



Spot the Fake Photo

People sometimes cheat in our mobile app by taking a photo of another screen - a phone or laptop showing a picture - instead of taking a real photo of the real thing. Build something that can tell the two apart.

In one line: given one image, decide whether it is a REAL photo or a PHOTO OF A SCREEN (a "recapture").

The bar: aim for more than 95% accuracy on our held-out photos.

What to do

1. Get some data (about 30-60 minutes).

Use your phone. Take ~50 normal photos of real things, and ~50 photos of a screen or a printout showing a picture. Put them in two folders: real/ and screen/. More variety (lighting, angles, different screens) makes your model better.

2. Solve it your way - training a model is NOT required.

Use whatever works: a trained model, classic computer-vision / image-processing tricks, frequency analysis, or any algorithm you come up with. Figuring out the approach is the real test. Keep it small and fast - it will eventually run on a phone.

3. Wrap it in a one-line predictor.

A script we can run as: `python predict.py some_image.jpg -> 0.93` (a number from 0 to 1: 0 = real photo, 1 = photo of a screen). A starter `predict.py` is included - just fill it in.

What to send us

- > Your code (`predict.py` + however you trained it).
- > A short note - half a page is plenty: how you did it, your accuracy, and what you'd improve with more time.
- > Optional (impressive): a tiny live demo - a web page that uses the camera and shows the result live.

Required: report two numbers

- > Latency - how long `predict.py` takes per image, in milliseconds, and on what device (e.g. "~20 ms, laptop CPU"). It should feel instant. Faster is better.
- > Cost per image - roughly what it would cost to run at scale. On-device (runs free on the user's phone) vs a cloud server (give a rough \$ per 1,000 or per million images, and state your assumptions). Cheaper is better.

How we judge

- > We run your `predict.py` on our OWN held-out photos - we're looking for 95%+ accuracy.
- > We read your note for clear thinking and an HONEST accuracy number.
- > We look at your latency + cost-per-image, and the trade-off you made.
- > We prefer a small, fast, cheap, honest solution over a big complicated one.

Why this is interesting: there's no object to "recognise" here - the clue is subtle (screen texture, glare, slightly-off colours). You can't just ask an AI chatbot "is this a screen?" and trust the answer; you have to build and test something real. That's the point.

More experienced? Also tell us (briefly): how you'd keep it accurate as cheaters adapt, how you'd make it tiny and fast enough for a phone, and how you'd choose the cut-off score for flagging fraud.